

Functional Specification

Project Title: Gamified App For Student

Author: Shuai Jiang

Date: 12/10/2025

1. Introduction

1.1 Project Overview

This project is a mobile app designed to help students manage their studies and personal growth in a more engaging way. To keep students motivated, the app uses several game-like features. By completing personal goals and real-world activities, students can earn Points, unlock achievement Badges, and compete on staged Leaderboards. What makes this app different is its focus on the real world. Instead of encouraging more online chatting, its key goal is to provide students with a tool to practice and improve their face-to-face social skills.

1.2 Scope

In-Scope: The project will deliver a native Android application with the features outlined in this document, supported by a Python-based backend. Key functional areas include personal task management, a class timetable, a focus mode, a step counter, a Bluetooth-based social interaction module, and a comprehensive gamification system.

Out-of-Scope: This project will not include direct integration with university systems (e.g., Blackboard), real-time online chat functionality, or complex RPG mechanics like character classes or equipment.

2. User Personas

To guide the design, we consider the following target user:

Name: Steve

Bio: A first-year university student feeling overwhelmed. Tasks from different classes and personal life are disorganized. Alex finds it hard to stay motivated for long study sessions and feels a bit socially isolated on a large campus.

Goals:

Find a single place to organize all tasks.

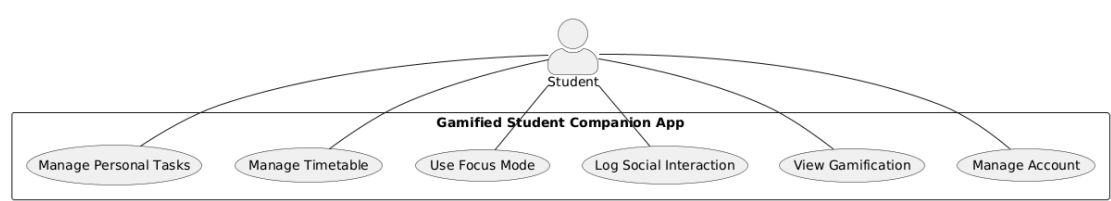
Build better study habits.

Find a low-pressure way to meet new people.

3. Use Cases

3.1 Use Case Diagram

The following diagram provides a high-level overview of the main interactions the "Student" actor can have with the application.



3.2 Use Case Prioritization (Master List)

This table provides a comprehensive overview of all planned functionalities, prioritized for development and estimated by difficulty, as requested by the supervisor.

- Priority:
 - Essential: Core features required for the Alpha/Beta version and screencast.
 - Good to Have: Features that add significant value but are secondary to the core loop.
 - Discretionary: Features to be considered only if time permits.
- Difficulty: 1 (Easy) to 5 (Very Complex).

ID	Use Case	Priority	Difficulty (1-5)
Account Management			

UC-AM-01	Register for a new account	Essential	2
UC-AM-02	Log in with email and password	Essential	1
UC-AM-03	Log out	Essential	1
UC-AM-04	View user profile	Essential	2
Productivity Tools			
UC-PD-01	Add a new personal task	Essential	2
UC-PD-02	Mark a task as complete	Essential	2
UC-PD-03	Add a new class to the weekly timetable	Good to Have	3
UC-PD-04	Receive a notification before a class starts	Good to Have	3
UC-PD-05	Start a focus session	Good to Have	3
UC-PD-06	Successfully complete a focus session	Good to Have	3

UC-PD-07	Fail or interrupt a focus session	Good to Have	3
Real-World Activities			
UC-RW-01	View daily step count	Discretionary	2
UC-RW-02	Enable or disable the social interaction feature	Essential	1
UC-RW-03	Mutually confirm a social interaction (Bluetooth)	Essential	5
Gamification Engine			
UC-GM-01	Receive points after completing a rewarded action	Essential	2
UC-GM-02	Lose points (Credits) after failing a challenge	Good to Have	2
UC-GM-03	Receive a notification upon leveling up	Good to Have	2
UC-GM-04	View the collection of all available badges	Good to Have	2

UC-GM-05	View the user's current rank on the Leaderboard	Essential	3
UC-GM-06	Browse and purchase virtual items with in-app credits	Discretionary	4

3.3 Master Use Case List

This list provides a comprehensive overview of all planned functionalities for the application.

Account Management

UC-AM-01: Register for a new account (Priority: **Essential** | Difficulty: **2**)

Actor: A new student.

Goal: To create a new account.

Flow:

1. A new user opens the app and selects "Register."
2. The system asks for an email and a password.
3. The user provides the details and submits the form.
4. The system creates a new user account and automatically logs the user in.

UC-AM-02: Log in with email and password (Priority: **Essential** | Difficulty: **1**)

Actor: A registered student.

Goal: Access an existing one.

Flow:

1. A returning user opens the app and selects "Login."
2. The system asks for their registered email and password.
3. The user provides the credentials and submits.
4. The system verifies the credentials and grants the user access to the main application.

UC-AM-03: Log out (Priority: **Essential** | Difficulty: **1**)

Actor: A logged-in Student.

Goal: To securely end the current session and return to the login screen.

Flow:

1. The user navigates to the "Profile" or "Settings" screen.
2. The user taps the "Log Out" button.
3. The system asks for confirmation (e.g., "Are you sure you want to log out?").
4. The user confirms.
5. The system clears the user's session data from the device and navigates to the "Login" screen.

UC-AM-04: View user profile (showing username, level, points, badges) (Priority: **Essential** | Difficulty: 2)

Actor: A logged-in student.

Goal: To check their overall progress, including gamification stats and achievements.

Flow:

1. The user navigates to the "Profile" screen (e.g., by tapping a profile icon or menu item).
2. The App UI sends a request to the backend for the user's profile data.
3. The backend retrieves the user's data from the database (including username, total XP, current level, and a list of earned badge IDs).
4. The backend returns this data to the App UI.
5. The App UI displays the user's name, current Level, a progress bar showing XP towards the next level, and the collection of all Badges they have earned.

Productivity Tools(The following text will use "PD" instead of "Personal Task" for distinction.)

UC-PD-01: Add a new personal task (Priority: **Essential** | Difficulty: 2)

Actor: A logged-in student.

Goal: To allow a logged-in user to add a new task to their To-Do list.

Flow:

1. The user navigates to the "To-Do List".
2. The user taps "Add Task".
3. The system presents a form with details (title, due date).
4. The user fills in the details and saves.
5. The new task then appears on their list.

UC-PD-02: Mark a task as complete (Priority: **Essential** | Difficulty: 2)

Actor: A logged-in student.

Goal: To mark a task from the To-Do list as finished and receive points.

Flow:

1. The user is on the "To-Do List" screen.
2. The user taps the checkbox next to a specific task.

3. The App UI sends a "Mark Complete" request to the backend for that task.
4. The backend verifies the action, marks the task as complete in the database, and triggers the gamification logic (see UC-GM-01).
5. The backend returns a success message (e.g., "Task complete! +10 Points").
6. The App UI updates to show the task as completed (e.g., greyed out or moved) and displays the points notification.

UC-PD-03: Add a new class to the weekly timetable (Priority: **Good to Have** | Difficulty: **3**)

Actor: A logged-in Student.

Goal: To allow a logged-in user to add their class schedule.

Flow:

1. The user navigates to the "Timetable" screen.
2. The user taps an "Add" button.
3. The system presents a form with fields for Class Name, Location, Day of the Week, Start Time, and End Time.
4. The user provides the details and submits the form.
5. The system saves the new class to the database and refreshes the timetable view to include the new entry.

UC-PD-04: Receive a notification before a class starts (Priority: **Good to Have** | Difficulty: **3**)

Actor: A logged-in Student.

Goal: To be reminded of an upcoming class from the timetable.

Flow:

1. The user has previously added a class to their timetable (see UC-PD-03).
2. The app's background service (or a scheduled local notification on the device) checks the system time against the saved timetable.
3. At a user-defined interval (e.g., 10 minutes) before the class's Start Time, the system issues a push notification to the user's device (e.g., "Class 'COMP-101' starts in 10 minutes in Room 204").

UC-PD-05: Start a focus session with a user-defined duration (Priority: **Good to Have** | Difficulty: **3**)

Actor: A logged-in Student.

Goal: To begin a timed study session to earn points.

Flow:

1. The user navigates to the "Focus Mode" screen.

2. The system presents an interface to set a time duration (e.g., a slider or input field for 20 minutes, 1 hour, etc.).
3. The user selects their desired duration and taps the "Start Session" button.
4. The App UI starts a countdown timer and displays a non-interactive focus screen.
5. The app begins monitoring for phone usage (like switching apps) in the background.

UC-PD-06: Successfully complete a focus session to earn rewards (Priority: **Good to Have** | Difficulty: **3**)

Actor: A logged-in Student.

Goal: To finish a started focus session without interruption and receive points.

Flow:

1. The user is currently in an active Focus session (started via UC-PD-05).
2. The user does not switch to other applications or unlock their phone for the entire duration of the timer.
3. When the timer completes, the App UI displays a "Success!" message.
4. The App UI sends a "Report Session Success" request to the backend, including the session's duration.
5. The backend verifies the success and triggers the gamification logic (see UC-GM-01) to calculate and award points.
6. The backend returns a success message with the points earned (e.g., "You earned 50 points!").
7. The App UI displays this message to the user.

UC-PD-07: Fail or interrupt a focus session and receive a penalty (Priority: **Good to Have** | Difficulty: **3**)

Actor: A logged-in Student.

Goal: To handle the case where a user breaks concentration, interrupts the session, and receives a penalty.

Flow:

1. The user is currently in an active focus session (started via UC-PD-05).
2. The user switches to another application (or leaves the focus screen) before the timer completes.
3. The app's background monitoring service detects this action.
4. The App UI immediately displays a "Failure!" message (e.g., "Session Interrupted!").
5. The App UI sends a "Report Session Failure" request to the backend.
6. The backend verifies the failure and triggers the gamification logic (see UC-GM-02) to calculate and apply the penalty (e.g., subtracting a small

- number of points).
7. The backend returns a failure message with the points lost (e.g., "Session Failed. -5 Points.").
 8. The App UI displays this message to the user.

Real-World Activities

UC-RW-01: View daily step count (Priority: **Discretionary** | Difficulty: **2**)

Actor: A logged-in Student.

Goal: To see their daily physical activity and earn potential rewards.

Flow:

1. The user navigates to the "Profile" or a dedicated "Health" screen.
2. The app requests permission to access the phone's native health service (e.g., Google Fit on Android).
3. If permission is granted, the app reads the current daily step count.
4. The system displays the step count to the user (e.g., "Steps Today: 6,000 / 10,000").
5. If the step count crosses a daily goal, the app triggers the gamification logic (see UC-GM-01) to award points.

UC-RW-02: Enable or disable the social interaction feature (Priority: **Essential** | Difficulty: **1**)

Actor: A logged-in Student.

Goal: To control their privacy by opting in or out of the Bluetooth social feature.

Flow:

1. The user navigates to the "Settings" screen.
2. The user finds the "Offline Social Feature" toggle (default is OFF).
3. The user taps the toggle to "ON".
4. The system presents a pop-up explaining what the feature does, what permissions (Bluetooth, Location) it needs, and affirms that it is anonymous.
5. The user accepts. The system requests the necessary OS permissions.
6. The feature is now enabled. The user can disable it at any time by tapping the toggle again.

UC-RW-03: Mutually confirm a social interaction with another user via Bluetooth (Priority: **Essential** | Difficulty: **5**)

Actor: Two students (Student A, Student B).

Goal: To securely verify a real-world interaction and for both users to receive points.

Precondition: Both users are logged in, have the app open, and have enabled the social feature.

Flow:

1. Student A and Student B meet and have a conversation in the real world.
2. Both students open the social feature in the app. The app uses Bluetooth to detect proximity.
3. Both students tap a "Confirm Interaction" button on their respective devices to mutually verify the interaction.
4. The system registers the confirmation and awards both students points.

Gamification Engine

UC-GM-01: Receive points after completing a rewarded action (Priority: **Essential** | Difficulty: **2**)

Actor: A logged-in Student.

Goal: To have the user's points (XP) and level progress updated automatically after completing a valuable action.

Flow:

1. A user successfully completes a rewarded action (e.g., marks a task complete , successfully finishes a focus session).
2. The App UI sends a request to the backend API corresponding to that action .
3. The backend verifies the action is valid.
4. The backend logic looks up the point value for that action.
5. The backend adds these points to the user's total XP and weekly XP in the database.
6. The backend checks if this new XP total crosses the threshold for a level up (see UC-GM-03).
7. The backend returns a success message to the App UI, including the number of points awarded.
8. The App UI displays a positive notification (e.g., "+10 Points!").

UC-GM-02: Lose points after failing a challenge (Priority: **Good to Have** | Difficulty: **2**)

Actor: A logged-in Student.

Goal: To apply a penalty to the user's points after failing a defined challenge.

Flow:

1. A user fails a challenge (e.g., interrupts a Focus Mode session, as described in UC-PD-07).
2. The App UI sends a "Report Session Failure" request to the backend API.
3. The backend verifies the failure is valid.
4. The backend logic looks up the penalty value for that failure.
5. The backend subtracts these Points from the user's currency total in the

database.

6. The backend returns a failure message to the App UI, including the number of Credits lost.
7. The App UI displays a notification (e.g., "Session Failed. -5 Credits.").

UC-GM-03: Receive a notification upon leveling up (Priority: **Good to Have** | Difficulty: 2)

Actor: A logged-in Student.

Goal: To be notified of their level progression when they have earned enough XP.

Flow:

1. A user receives XP after a rewarded action (as part of UC-GM-01).
2. The backend logic checks if the user's new total XP meets or exceeds the threshold for the next level.
3. If it does, the backend updates the user's level (e.g., from 2 to 3) in the database.
4. The backend then returns a special "Level Up!" event or message to the App UI, along with the points earned.
5. The App UI receives this event and displays a prominent, celebratory notification or animation (e.g., "Congratulations! You've reached Level 3!").

UC-GM-04: View the collection of all available badges (Priority: **Good to Have** | Difficulty: 2)

Actor: A logged-in Student.

Goal: To see all possible achievements (badges) and their progress toward them.

Flow:

1. The user navigates to their "Profile" screen or a dedicated "Achievements" section.
2. The App UI requests the full badge list from the backend.
3. The backend retrieves the list of all possible badges, along with the specific badges the user has already earned.
4. The App UI displays a gallery of all badges. Earned badges are shown in full color, while unearned badges are greyed out and display the criteria required to unlock them (e.g., "Focus for 50 hours").

UC-GM-05: View the user's current rank within their league on the Leaderboard (Priority: **Essential** | Difficulty: 3)

Actor: A logged-in Student.

Goal: To allow a logged-in user to see their competitive rank within their current league .

Flow:

1. The user navigates to the "Leaderboard" screen.
2. The App UI sends a request to the backend for the leaderboard data specific to the user's league.
3. The backend retrieves the list of users in that league, sorted by their current weekly points.
4. The backend returns this ranked list to the App UI.
5. The App UI displays the ranked list and highlights the user's own entry for easy identification.

UC-GM-06: Browse and purchase virtual items with in-app credits (Priority:

Discretionary | Difficulty: **4**)

Actor: A logged-in Student.

Goal: To spend earned in-app "Credits" on virtual rewards (e.g., new app themes, "Streak Freeze" items).

Flow:

1. The user navigates to the "Shop" screen.
2. The App UI requests the list of available shop items from the backend.
3. The backend returns the list of items (e.g., "New App Theme") and their prices in "Credits."
4. The user taps the "Buy" button on an item they can afford.
5. The App UI asks for confirmation (e.g., "Spend 100 Credits on 'New App Theme'?").
6. The user confirms the purchase.
7. The App UI sends a "Purchase Item" request to the backend.
8. The backend verifies the user has enough "Credits."
9. The backend subtracts the cost from the user's "Credits" total and adds the item to the user's "inventory" in the database.
10. The backend returns a success message.
11. The App UI updates the user's "Credits" total and shows the item as "Owned" or "Active."

3.4 Detailed Use Cases & Sequence Diagrams

The following sections describe the flow of key use cases, starting with fundamental actions and highlighting the key innovative feature with a sequence diagram.

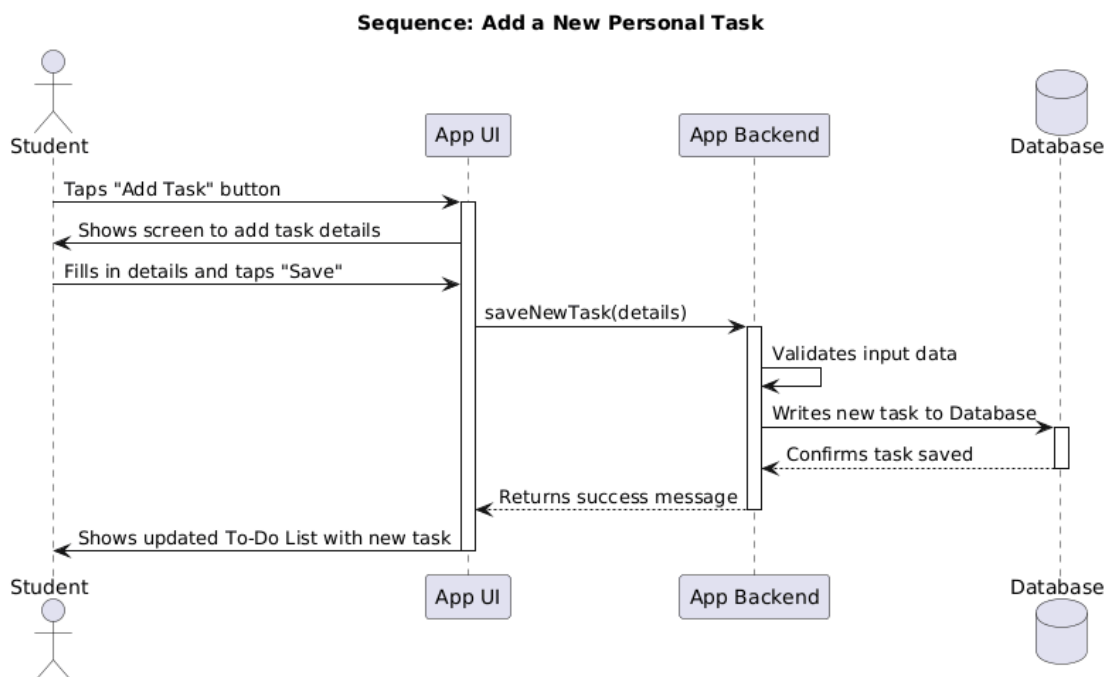
3.4.1 Fundamental Use Cases

- **Use Case:** User Registration and Login (Priority: **Essential** | Difficulty: **2**)
 - **Actor:** A new or returning Student.

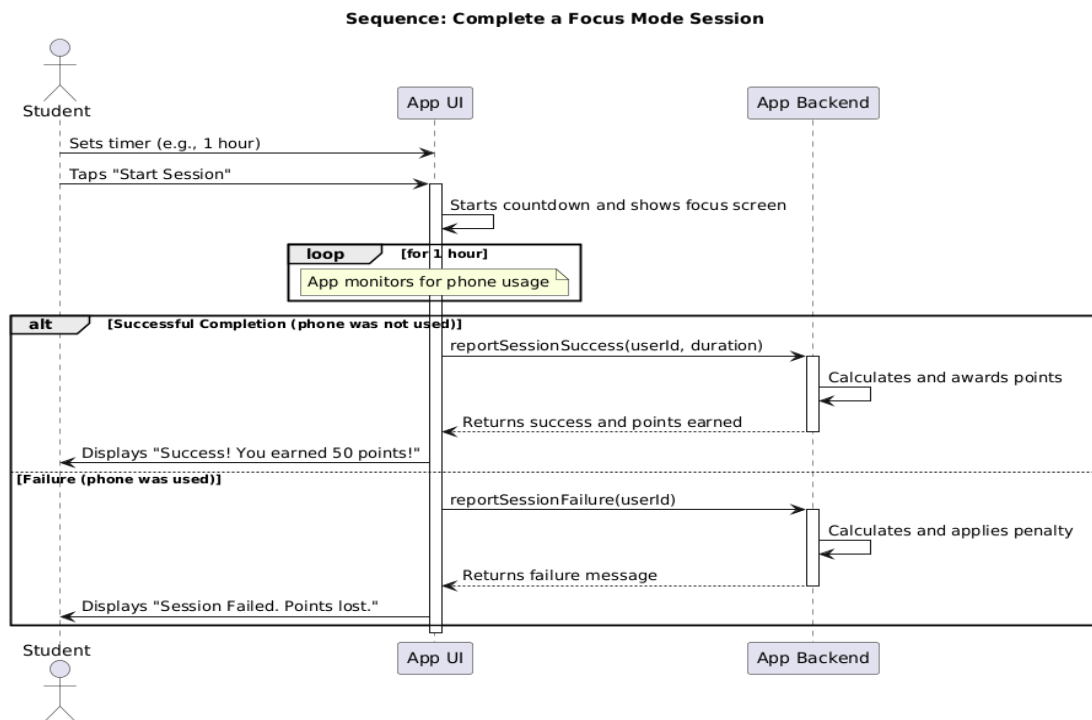
- **Goal:** To create a new account or access an existing one.
- **Registration Flow:**
 1. A new user opens the app and selects "Register."
 2. The system asks for an email and a password.
 3. The user provides the details and submits the form.
 4. The system creates a new user account and automatically logs the user in.
- **Login Flow:**
 - 1.1.1. A returning user opens the app and selects "Login."
 - 1.1.2. The system asks for their registered email and password.
 - 1.1.3. The user provides the credentials and submits.
 - 1.1.4. The system verifies the credentials and grants the user access to the main application.

3.4.2 Core Feature Use Cases

- **Use Case:** Add a New Personal Task (Priority: **Essential** | Difficulty: **2**)
 - **Goal:** To allow a logged-in user to add a new task to their To-Do list.
 - **Flow:** The user navigates to the "To-Do List," taps "Add Task," fills in the details (title, due date), and saves. The new task then appears on their list.



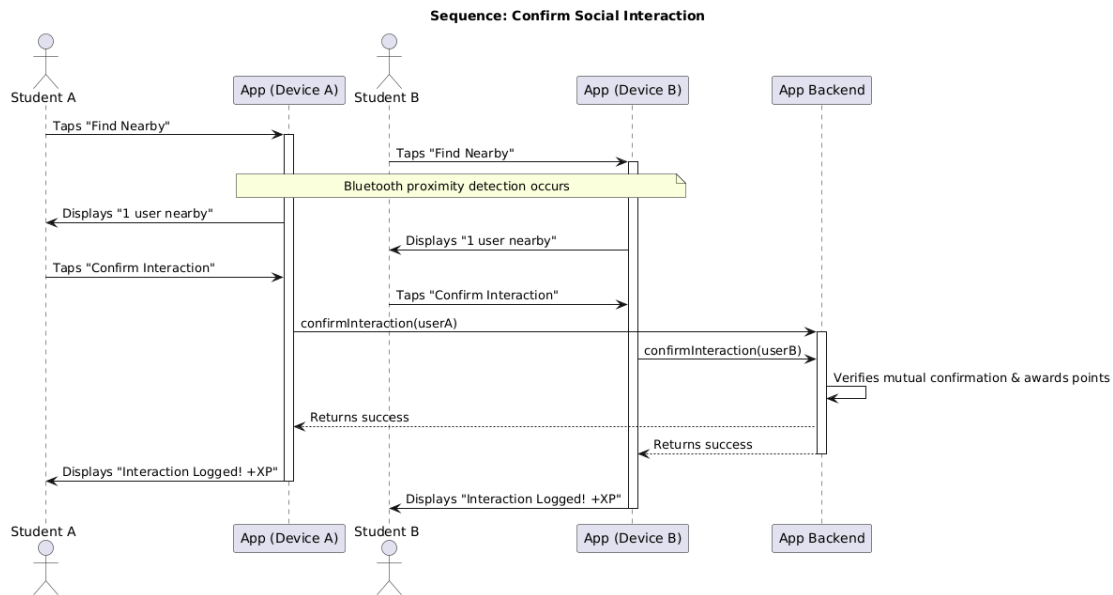
- **Use Case:** Complete a Focus Mode Session (Priority: **Good to Have** | Difficulty: **3**)
 - **Goal:** To help a logged-in user study without distraction and earn points.
 - **Flow:** The user navigates to the "Focus Mode," sets a timer, and starts the session. If they complete the session without switching apps, they are awarded points. If they fail, they receive a small point penalty.



3.4.3 Key Innovative Use Case

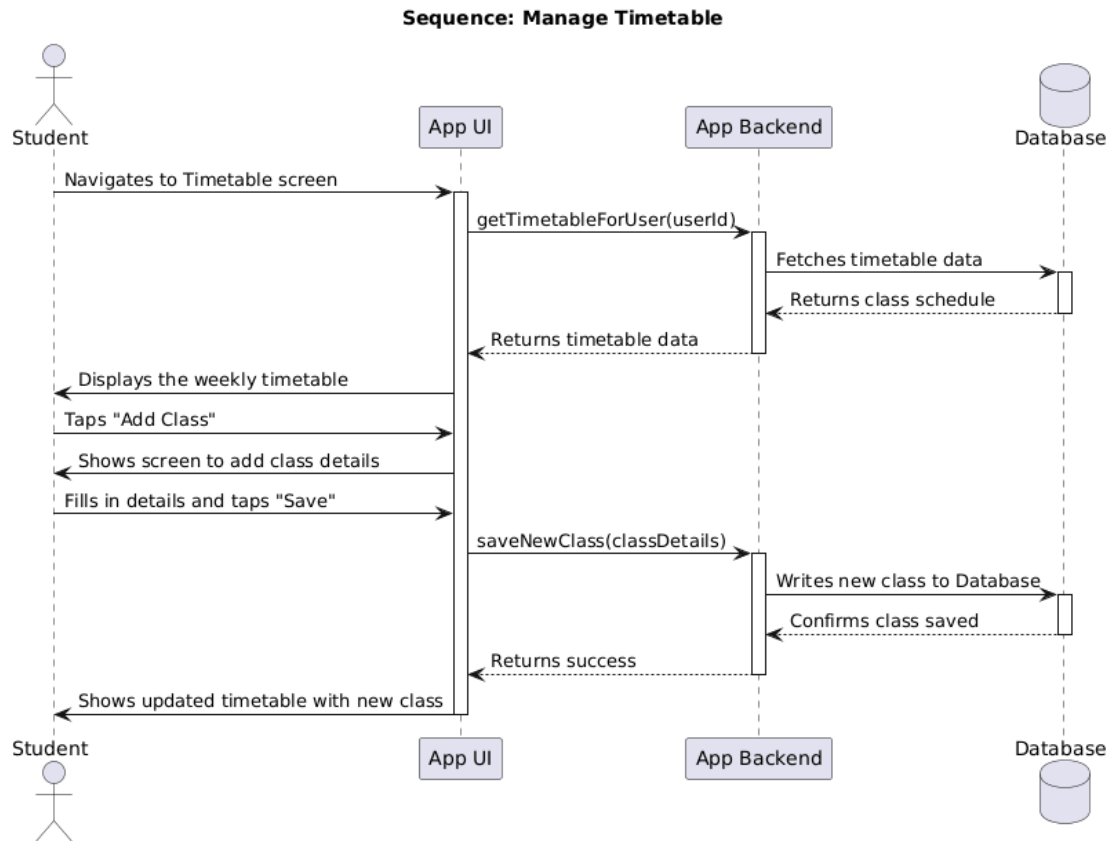
- **Use Case:** Confirm a Face-to-Face Social Interaction (Priority: **Essential** | Difficulty: **5**)
 - **Actor:** Two students (Student A, Student B).
 - **Precondition:** Both users are logged in, have the app open, and have enabled the social feature.
 - **Main Flow:**
 1. Student A and Student B meet and have a conversation in the real world.
 2. Both students open the social feature in the app. The app uses Bluetooth to detect proximity.
 3. Both students tap a "Confirm Interaction" button on their respective devices to mutually verify the interaction.

- The system registers the confirmation and awards both students points.



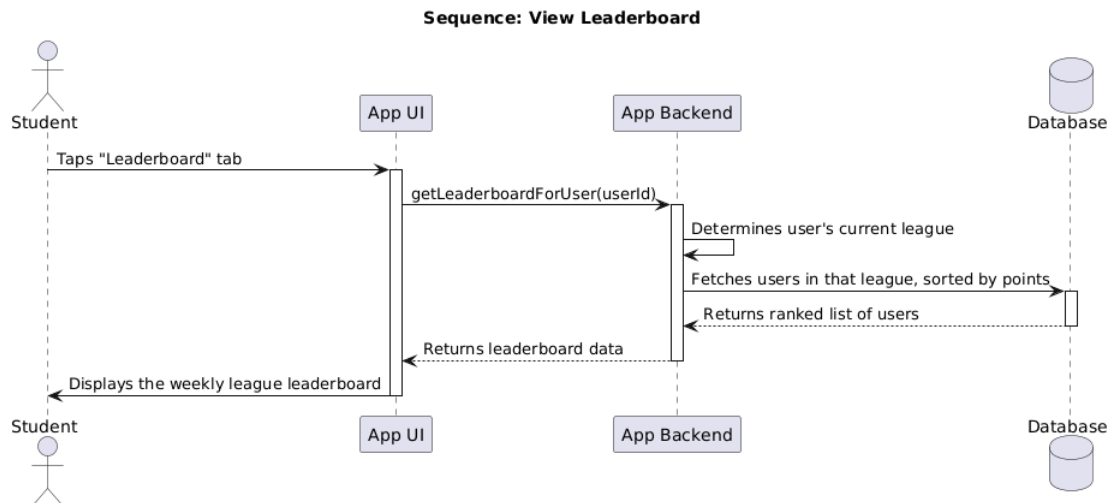
3.4.4 Manage Weekly Timetable

- Use Case:** To allow a logged-in user to add and view their class schedule.
(Priority: **Good to Have** | Difficulty: **3**)
- Actor:** A logged-in Student.
- View Timetable Flow:**
 - The user navigates to the "Timetable" screen.
 - The system requests the user's schedule from the backend.
 - The system displays the classes for the current week in a calendar view.
- Add Class Flow:**
 - From the Timetable screen, the user taps an "Add" button.
 - The system presents a form with fields for Class Name, Location, Day of the Week, Start Time, and End Time.
 - The user provides the details and submits the form.
 - The system saves the new class to the database and refreshes the timetable view to include the new entry.



3.4.5 View Leaderboard

- **Goal:** To allow a logged-in user to see their competitive rank within their current league. (Priority: **Essential** | Difficulty: **3**)
- **Actor:** A logged-in Student.
- **Flow:**
 1. The user navigates to the "Leaderboard" screen.
 2. The system requests the leaderboard data for the user's league from the backend.
 3. The system displays a ranked list of users based on the current week's points.
 4. The system highlights the user's own entry within the list for easy identification.



3.4.6 View User Profile

- **Goal:** To allow a logged-in user to check their overall progress and achievements. (Priority: **Essential** | Difficulty: **2**)
- **Actor:** A logged-in Student.
- **Flow:**
 1. The user navigates to the "Profile" screen.
 2. The system requests the user's complete profile data from the backend.
 3. The system displays the user's name, current Level, a progress bar showing XP towards the next level, and the collection of all Badges they have earned.

